



РЭУ.РФ
РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ Г.В. ПЛЕХАНОВА

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Лис Андрей Игоревич
(подпись) (фамилия, имя, отчество)

Группы ИПо-32

Руководитель практической подготовки от профильной организации

Преподаватель _____
(должность)

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

МП «26» июня 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТОВ	5
1.1. Сбор информации	5
1.2. Разработка Технического Задания.	6
РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ	9
2.1. Процесс разработки программного обеспечения	9
РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ	17
3.1. Процесс отладки	17
РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	18
4.1. Процесс тестирования	18
РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ...	23
ЗАКЛЮЧЕНИЕ.....	26
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	28

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППСЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: Разработка программного обеспечения; выполнение отладки программного модуля с использованием специализированных программных средств; осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; уметь производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент;
2. Выполнять интеграцию модулей в программное обеспечение;
3. Выполнять отладку программного модуля с использованием специализированных программных средств;

4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.
5. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования
6. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
7. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
8. Планировать и реализовывать собственное профессиональное и личностное развитие;
9. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
10. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
11. Использовать информационные технологии в профессиональной деятельности
12. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТОВ

1.1. Сбор информации

Постановка задач:

Разработать настольное приложение для управления задачами (Task Manager) с возможностью добавления, редактирования, удаления, фильтрации, поиска и сохранения задач в файлы JSON и XML.

Состояние «AS IS» (Как есть сейчас):

Пользователь управляет своими задачами вручную, используя бумажные списки задач (блокноты, стикеры), текстовые файлы без структуры и Excel таблицы с ручным вводом. Проблемы текущего подхода заключаются в отсутствии единого места хранения всех задач, невозможности быстро отфильтровать задачи по статусу, отсутствии функции поиска по ключевым словам, неудобстве редактирования и удаления записей, риске потери данных и отсутствии автоматической сортировки.

Состояние «TO BE» (Как будет после внедрения):

После разработки приложения Task Manager пользователь получает возможность полноценной работы с задачами. При добавлении задачи пользователь открывает приложение Task Manager, нажимает кнопку "Добавить задачу", заполняет форму (название, описание, приоритет, срок, статус), после чего задача автоматически сохраняется. Для просмотра задач все они отображаются в таблице с удобным визуальным представлением. Фильтрация осуществляется путем выбора статуса из выпадающего списка, после чего система показывает только задачи с выбранным статусом. Поиск реализован через ввод текста в строку поиска с мгновенным поиском по названию или описанию. Редактирование и удаление выполняется через

выбор задачи и нажатие кнопки "Редактировать" или "Удалить" с последующим подтверждением операции. Сохранение и загрузка данных осуществляется путем записи всех задач в JSON или XML файл с возможностью загрузки при следующем запуске.

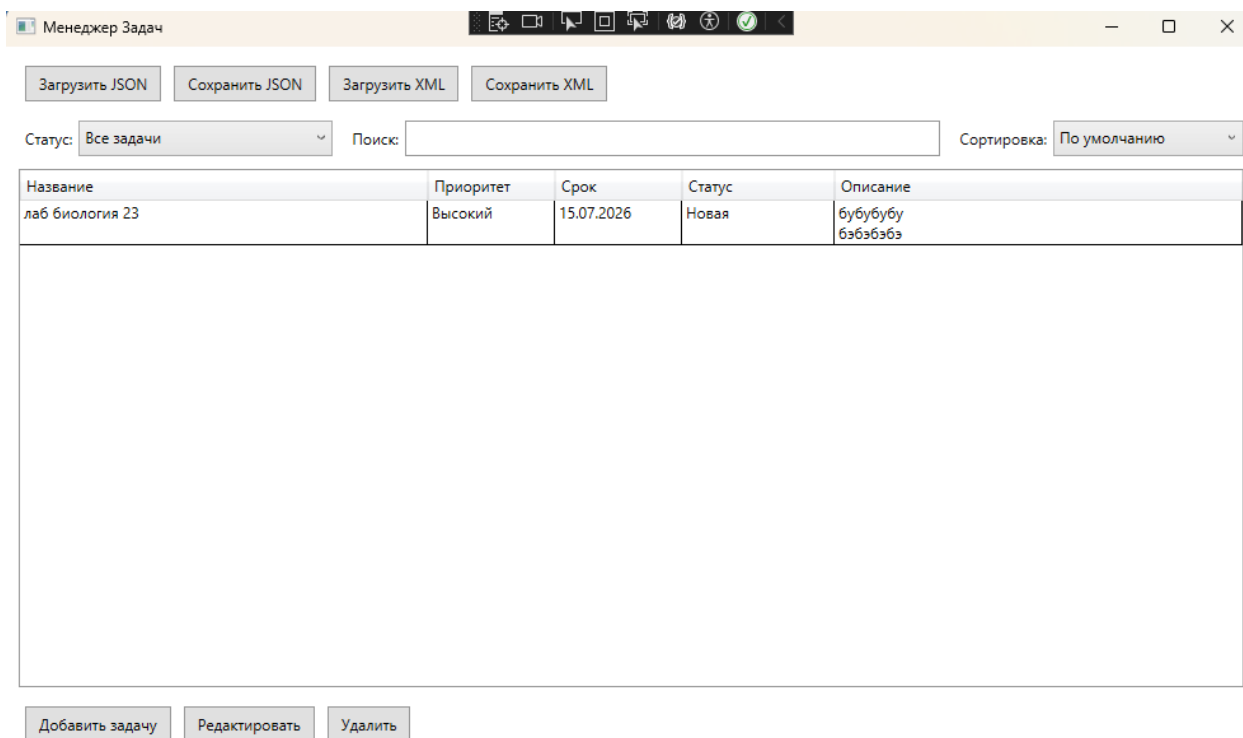


рисунок 1. главное окно

1.2. Разработка Технического Задания.

ГОСТ 19.201-78 — единая система программной документации (ЕСПД), используется для описания отдельных программ и программных модулей. Определяет требования к содержанию и оформлению технического задания.

Структура Технического Задания:

1. Общие сведения:

- Наименование программы: Task Manager
- Область применения: персональное управление задачами
- Основание для разработки: задание на учебную практику

2. Назначение и цели:

Цель: создание настольного приложения для управления задачами с возможностью создания, редактирования, фильтрации, поиска

и сохранения

3. Функциональные требования:

- Добавление задачи (название, описание, приоритет, срок, статус)
- Просмотр списка всех задач
- Фильтрация задач по статусу
- Поиск задач по названию или описанию
- Редактирование существующих задач
- Удаление задач
- Сохранение в JSON файл
- Загрузка из JSON файла
- Сохранение в XML файл
- Загрузка из XML файла
- Сортировка по приоритету (дополнительно)
- Сортировка по сроку выполнения (дополнительно)
- Отметка важных задач (дополнительно)
- Статистика выполнения (дополнительно)

4. Технические требования:

- Платформа: Windows 10/11
- Фреймворк: .NET 9.0
- Язык: C# 12
- Технология UI: WPF
- Хранение: JSON и XML файлы

5. Требования к архитектуре

- Библиотека классов (TaskManagerLibrary.dll) — бизнес-логика
- WPF приложение (TaskManagerApp.exe) — интерфейс
- Тестовый проект (TaskManagerLibrary.Tests) — unit-тесты
- Разделение логики и представления

6. Требования к надежности

- Валидация входных данных
- Обработка ошибок при работе с файлами

- Подтверждение критичных операций
- Сохранность данных

7. Требования к тестированию

- Unit-тесты для всех классов библиотеки
- Фреймворк: xUnit
- Покрытие всех публичных методов

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1. Процесс разработки программного обеспечения

Выбор модели разработки:

Для данного проекта выбрана итерационная модель разработки. Обоснование выбора заключается в том, что итерационная модель подходит для учебных проектов, так как позволяет постепенно добавлять функциональность, каждая итерация тестируется отдельно, можно вносить изменения на основе результатов, получаем рабочий прототип на ранних стадиях и обеспечивается гибкость для одного разработчика.

Итерации разработки включали создание базовых классов (TaskItem, TaskStatus, TaskPriority), разработку менеджера задач (TaskManager с LINQ запросами), добавление работы с файлами (TaskRepository для JSON/XML), создание WPF интерфейса (MainWindow, диалоги), написание unit тестов и отладку с оптимизацией.

Инструменты разработки включали IDE Microsoft Visual Studio 2022 Community Edition, язык программирования C# 12, целевую платформу .NET 9.0, технологию пользовательского интерфейса WPF (Windows Presentation Foundation), фреймворк тестирования xUnit и опционально систему контроля версий Git.

Используемые библиотеки включали System.Collections.Generic для работы с коллекциями, System.Linq для LINQ запросов (фильтрация, поиск, сортировка), System.Text.Json для сериализации JSON, System.Xml.Serialization для работы с XML, System.ComponentModel.DataAnnotations для валидации, System.Windows для WPF компонентов и xUnit для тестирования.

Среда программирования Visual Studio 2022:

Visual Studio представляет собой интегрированную среду разработки от Microsoft, которая включает редактор кода с IntelliSense (автодополнение),

отладчик с breakpoints, дизайнер форм WPF (XAML), менеджер пакетов NuGet, Test Explorer для unit тестов и Solution Explorer для управления проектами.

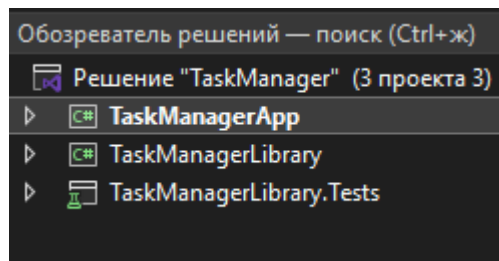


рисунок 2. обозреватель решений

Диаграмма классов системы:

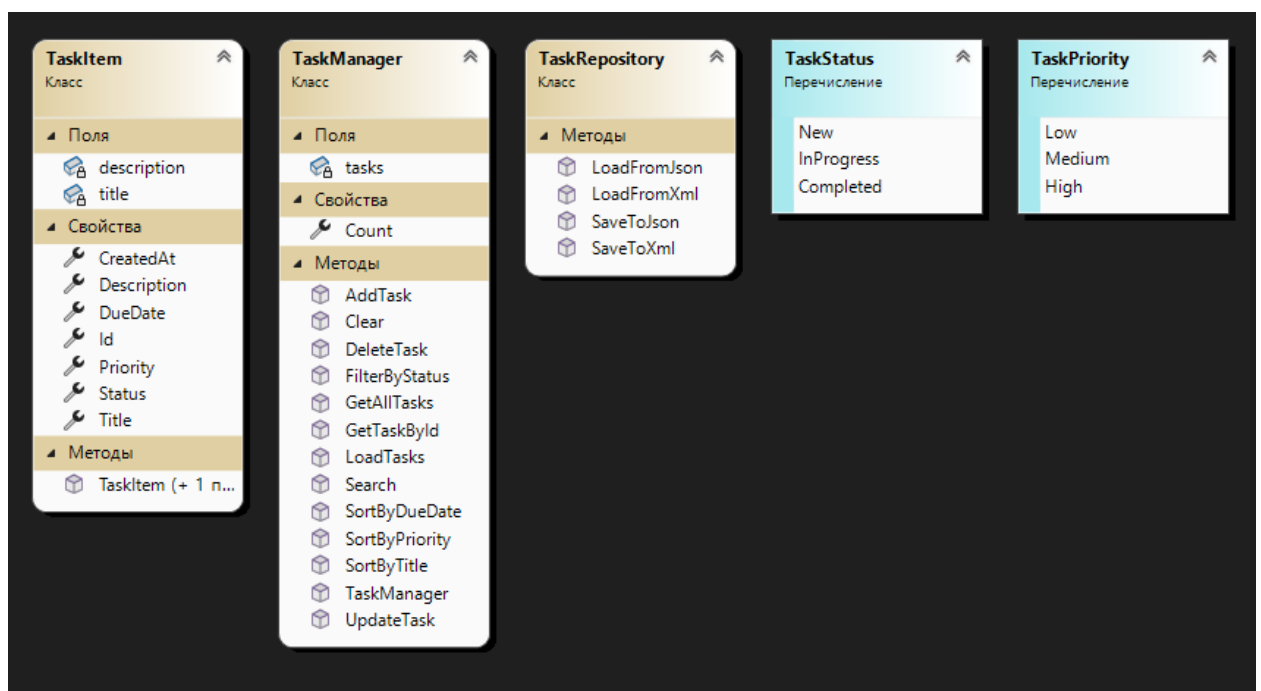


рисунок 3. диаграмма классов

Структура классов системы представлена перечислением TaskStatus для статусов задачи (New, InProgress, Completed), перечислением TaskPriority для приоритетов (Low, Medium, High), классом TaskItem в качестве модели задачи, классом TaskManager как менеджером для CRUD операций и LINQ запросов, и классом TaskRepository для работы с файлами JSON и XML.

Разработка программного модуля согласно требованиям:

Проект 1: TaskManagerLibrary (Class Library)

Файл: TaskItem.cs представляет модель задачи с полями Id (уникальный идентификатор типа Guid), Title (название длиной от 1 до 200 символов), Description (описание до 1000 символов), Priority (приоритет типа TaskPriority), DueDate (срок выполнения типа DateTime), Status (текущий статус типа TaskStatus), CreatedAt (дата создания типа DateTime) и IsImportant (признак важности типа bool как опциональное поле).

```
public class TaskItem
{
    Ссылка 12
    public Guid Id { get; set; }

    private string title = string.Empty;

    [Required(ErrorMessage = "Название задачи обязательно")]
    [StringLength(200, MinimumLength = 1, ErrorMessage = "Название должно быть от 1 до 200 символов")]
    Ссылка 24
    public string Title
    {
        get => title;
        set
        {
            if (string.IsNullOrEmpty(value))
                throw new ArgumentException("Название задачи не может быть пустым");
            if (value.Length > 200)
                throw new ArgumentException("Название задачи не может превышать 200 символов");
            title = value.Trim();
        }
    }

    private string description = string.Empty;

    [StringLength(1000, ErrorMessage = "Описание не может превышать 1000 символов")]
    Ссылка 8
    public string Description
    {
        get => description;
        set
        {
            if (value != null && value.Length > 1000)
                throw new ArgumentException("Описание не может превышать 1000 символов");
            description = value?.Trim() ?? string.Empty;
        }
    }
}
```

рисунок 4. Код класса taskitem.cs

Файл: TaskManager.cs представляет менеджер задач с методами CRUD операций, включающими AddTask(TaskItem task) для добавления с валидацией, UpdateTask(Guid id, TaskItem updated) для обновления, DeleteTask(Guid id) для удаления по ID, GetTaskById(Guid id) для получения задачи по ID и GetAllTasks для получения всех задач. Также реализованы LINQ запросы, такие как FilterByStatus(TaskStatus status) с использованием tasks.Where(t => t.Status == status), Search(string query) для поиска по названию или описанию

через `tasks.Where(t=>t.Title.Contains(query, StringComparison.OrdinalIgnoreCase) || t.Description.Contains(query, StringComparison.OrdinalIgnoreCase))`, `SortByPriority` через `tasks.OrderByDescending(t => t.Priority)`, `SortByDueDate` через `tasks.OrderBy(t => t.DueDate)`, `SortByTitle` через `tasks.OrderBy(t => t.Title)`, а также `GetStatistics` для получения статистики (всего задач, завершено, просрочено).

```
// фильтрации
Ссылка 3
public IEnumerable<TaskItem> FilterByStatus(TaskStatus? status)
{
    if (status == null)
        return tasks;

    return tasks.Where(t => t.Status == status.Value);
}

Ссылка 4
public IEnumerable<TaskItem> Search(string searchText)
{
    if (string.IsNullOrEmpty(searchText))
        return tasks;

    searchText = searchText.ToLower();
    return tasks.Where(t =>
        t.Title.ToLower().Contains(searchText) ||
        t.Description.ToLower().Contains(searchText));
}

Ссылка 2
public IEnumerable<TaskItem> SortByPriority()
{
    return tasks.OrderByDescending(t => t.Priority);
}

Ссылка 2
public IEnumerable<TaskItem> SortByDueDate()
{
    return tasks.OrderBy(t => t.DueDate);
}

Ссылка 2
public IEnumerable<TaskItem> SortByTitle()
{
    return tasks.OrderBy(t => t.Title);
}

//
```

рисунок 5. код taskmanager.cs

Файл: `TaskRepository.cs` представляет репозиторий для работы с файлами и содержит методы для JSON, включая `SaveToJson(List<TaskItem> tasks, string filePath)` с использованием `System.Text.Json.JsonSerializer` и `LoadFromJson(string filePath)` возвращающий `List<TaskItem>` с обработкой ошибок (файл не найден, неверный формат). Также реализованы методы для XML,

такие как `SaveToXml(List<TaskItem> tasks, string filePath)` с использованием `System.Xml.Serialization.XmlSerializer` и `LoadFromXml(string filePath)` возвращающий `List<TaskItem>` с обработкой исключений.

```
public void SaveToXml(string filePath, IEnumerable<TaskItem> tasks)
{
    if (string.IsNullOrEmpty(filePath))
        throw new ArgumentException("Путь к файлу не может быть пустым", nameof(filePath));

    try
    {
        var serializer = new XmlSerializer(typeof(List<TaskItem>));
        using var writer = new StreamWriter(filePath);
        serializer.Serialize(writer, tasks.ToList());
    }
    catch (Exception ex)
    {
        throw new InvalidOperationException($"Ошибка при сохранении в XML: {ex.Message}", ex);
    }
}

// загрузка из XML
Ссылка 3
public IEnumerable<TaskItem> LoadFromXml(string filePath)
{
    if (string.IsNullOrEmpty(filePath))
        throw new ArgumentException("Путь к файлу не может быть пустым", nameof(filePath));

    if (!File.Exists(filePath))
        throw new FileNotFoundException($"Файл не найден: {filePath}");

    try
    {
        var serializer = new XmlSerializer(typeof(List<TaskItem>));
        using var reader = new StreamReader(filePath);
        var tasks = serializer.Deserialize(reader) as List<TaskItem>;
        return tasks ?? new List<TaskItem>();
    }
    catch (InvalidOperationException ex)
    {
        throw new InvalidOperationException($"Ошибка десериализации XML: {ex.Message}", ex);
    }
    catch (Exception ex)
    {
        throw new InvalidOperationException($"Ошибка при загрузке из XML: {ex.Message}", ex);
    }
}
```

рисунок 6. код taskrepository.cs

Проект 2: TaskManagerApp (WPF Application)

Файл: `MainWindow.xaml` содержит XAML разметку главного окна с элементами интерфейса, включающими `DataGrid` для отображения списка задач с колонками название, приоритет, срок выполнения, статус и важность (если включена). Панель кнопок управления содержит кнопки добавить задачу, редактировать задачу и удалить задачу. Панель фильтрации и поиска включает `ComboBox` для выбора статуса, `TextBox` для поискового запроса и

ComboBox для критерия сортировки. Панель работы с файлами содержит кнопки сохранить в JSON, загрузить из JSON, сохранить в XML и загрузить из XML.

Файл: MainWindow.xaml.cs содержит логику главного окна с основными методами. Метод RefreshTaskList обновляет список с применением фильтров, применением поиска, применением сортировки и обновлением DataGrid. Метод AddButton_Click отвечает за добавление задачи, EditButton_Click за редактирование, DeleteButton_Click за удаление с подтверждением, FilterComboBox_SelectionChanged обрабатывает смену фильтра, SearchTextBox_TextChanged обрабатывает ввод текста поиска, SortComboBox_SelectionChanged обрабатывает смену сортировки, SaveJsonButton_Click выполняет сохранение в JSON, LoadJsonButton_Click загрузку из JSON, SaveXmlButton_Click сохранение в XML и LoadXmlButton_Click загрузку из XML.

```
private void RefreshTaskList()
{
    IEnumerable<TaskItem> tasks = manager.GetAllTasks();

    if (StatusFilterComboBox.SelectedIndex > 0)
    {
        TaskManagerLibrary.TaskStatus? status = StatusFilterComboBox.SelectedIndex switch
        {
            1 => TaskManagerLibrary.TaskStatus.New,
            2 => TaskManagerLibrary.TaskStatus.InProgress,
            3 => TaskManagerLibrary.TaskStatus.Completed,
            _ => null
        };
        tasks = manager.FilterByStatus(status);
    }
}
```

рисунок 7. Код mainwindow.xaml.cs

Файл: TaskDialog.xaml и TaskDialog.xaml.cs представляют диалог добавления и редактирования с элементами формы, включающими TextBox для названия, многострочный TextBox для описания, ComboBox для выбора приоритета, ComboBox для выбора статуса, DatePicker для срока выполнения, опциональный CheckBox для отметки важности, а также кнопки Сохранить и Отмена.

рисунок 8. диалоговое окно добавления задачи

Этапы разработки интерфейса включали проектирование, в ходе которого определялся состав окон, создавались эскизы интерфейса и определялись элементы управления. Затем следовало прототипирование с созданием XAML разметки, добавлением элементов (DataGrid, кнопки, фильтры) и настройкой Layout (Grid, StackPanel). Завершающим этапом была стилизация, включавшая применение светлой темы (белый фон, темный текст), настройку отступов, установку шрифтов и цветовое выделение важных задач.

Название	Приоритет	Срок	Статус	Описание
помыться	Низкий	30.06.2026	Новая	
поесть огурец	Средний	29.06.2026	Новая	помыть посолить откусить ну главное ювелирненько сол
лаб био 23	Высокий	01.07.2026	В процессе	буббуубуб

рисунок 9. Сортировка по приоритету

Название	Приоритет	Срок	Статус	Описание
поесть огурец	Средний	29.06.2026	Новая	помыть посолить откусить ну главное ювелирненько сол
помыться	Низкий	30.06.2026	Новая	
лаб био 23	Высокий	01.07.2026	В процессе	буббуубуб

рисунок 10. Сортировка по сроку

Статус: Все задачи Поиск:

Название	Приоритет	Срок
биол 32323232	Средний	01.07

рисунок 12. Поиск

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГР АММНЫХ СРЕДСТВ

3.1. Процесс отладки

Отладка представляет собой процесс поиска и исправления ошибок в коде. Для этого использовался встроенный отладчик Visual Studio 2022. Инструменты отладки включали Breakpoints (точки останова), которые останавливают выполнение на определенной строке, Step Over (F10) для выполнения текущей строки, Step Into (F11) для входа в вызываемый метод, Step Out (Shift+F11) для выхода из метода, Locals Window (окно с локальными переменными), Watch Window для отслеживания значений выражений, Call Stack для просмотра стека вызовов и Immediate Window для выполнения команд во время отладки.

Процесс проведения отладки осуществлялся следующим образом. На первом шаге устанавливался breakpoint путем открытия файла с кодом (например, TaskManager.cs), щелчка на левом поле редактора напротив нужной строки, после чего появлялась красная точка, обозначающая breakpoint. На втором шаге выполнялся запуск в режиме Debug нажатием F5 или кнопки "Start Debugging", при этом приложение запускалось в режиме отладки и выполнение останавливалось на breakpoint. Третий шаг включал анализ состояния, при котором в окне Locals отображались значения локальных переменных, в Watch добавлялись выражения для отслеживания, а желтая стрелка показывала текущую строку. На четвертом шаге выполнялось пошаговое выполнение с использованием F10 (Step Over) для выполнения текущей строки и F11 (Step Into) для входа в метод с наблюдением за изменением значений переменных. Пятый шаг предполагал поиск и исправление ошибок, при котором находилось неправильное значение, идентифицировался баг, останавливалась отладка (Shift+F5), исправлялся код и снова запускалась отладка для проверки.

РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1. Процесс тестирования

Тестирование представляет собой проверку соответствия реального поведения программы ожидаемому на тестовых данных. Для тестирования библиотеки `TaskManagerLibrary` использовался фреймворк `xUnit`, являющийся популярным фреймворком для unit тестирования в `.NET`. Проект `TaskManagerLibrary.Tests` (`xUnit Test Project`) имеет структуру, включающую `TaskItemTests.cs` для тестов класса `TaskItem`, `TaskManagerTests.cs` для тестов класса `TaskManager` и `TaskRepositoryTests.cs` для тестов класса `TaskRepository`.

Unit тесты для `TaskItem` (`TaskItemTests.cs`) включают тест `Constructor_WithValidData_CreatesTask`, проверяющий создание задачи с валидными данными, тест `Title_WhenEmpty_ThrowsException`, проверяющий что пустое название вызывает исключение, тест `Title_WhenTooLong_ThrowsException`, проверяющий что длинное название (более 200 символов) вызывает ошибку, тест `Description_WhenTooLong_ThrowsException`, проверяющий длинное описание (более 1000 символов), и тест `DefaultConstructor_SetsDefaultValues`, проверяющий значения по умолчанию.

```

Ссылка 0
public class TaskItemTests
{
    [Fact]
    Ссылка 0
    public void Constructor_WithValidData_CreatesTask()
    {
        var task = new TaskItem(
            "Test Task",
            "Test Description",
            TaskPriority.High,
            DateTime.Now.AddDays(5),
            TaskStatus.New
        );

        Assert.Equal("Test Task", task.Title);
        Assert.Equal("Test Description", task.Description);
        Assert.Equal(TaskPriority.High, task.Priority);
        Assert.Equal(TaskStatus.New, task.Status);
        Assert.NotEqual(Guid.Empty, task.Id);
    }

    [Fact]
    Ссылка 0
    public void Title_WhenEmpty_ThrowsException()
    {
        var task = new TaskItem();

        Assert.Throws<ArgumentException>(() => task.Title = "");
        Assert.Throws<ArgumentException>(() => task.Title = "   ");
    }

    [Fact]
    Ссылка 0
    public void Title_WhenTooLong_ThrowsException()
    {
        var task = new TaskItem();
        var longTitle = new string('a', 201);

        Assert.Throws<ArgumentException>(() => task.Title = longTitle);
    }
}

```

рисунок 13. тесты

Unit тесты для TaskManager (TaskManagerTests.cs) включают тест AddTask_WithValidTask_AddsToCollection, проверяющий добавление задачи, тест AddTask_WithNullTask_ThrowsException, проверяющий что null вызывает исключение, тест DeleteTask_WithExistingId_RemovesTask, проверяющий удаление существующей задачи, тест UpdateTask_WithValidData_UpdatesTask, проверяющий обновление задачи, тест GetTaskById_WithExistingId_ReturnsTask, проверяющий получение по ID, LINQ тест FilterByStatus_ReturnsOnlyMatchingTasks, который проверяет фильтрацию по статусу создавая задачи с разными статусами и проверяя фильтр, LINQ тест

Search_FindsTasksByTitle, проверяющий поиск по названию, LINQ тест Search_FindsTasksByDescription, проверяющий поиск по описанию, LINQ тест Search_IsCaseInsensitive, проверяющий регистронезависимый поиск, LINQ тест SortByPriority_ReturnsSortedList, который проверяет сортировку по приоритету (High → Medium → Low), LINQ тест SortByDueDate_ReturnsSortedList, проверяющий сортировку по дате, и LINQ тест SortByTitle_ReturnsSortedList, проверяющий алфавитную сортировку.

```
Ссылка 0
public class TaskManagerTests
{
    [Fact]
    Ссылка 0
    public void AddTask_WithValidTask_AddsToCollection()
    {
        var manager = new TaskManager();
        var task = new TaskItem("Test Task", "Description", TaskPriority.Medium, DateTime.Now, TaskStatus.New);

        manager.AddTask(task);

        Assert.Equal(1, manager.Count);
        Assert.Contains(task, manager.GetAllTasks());
    }

    [Fact]
    Ссылка 0
    public void AddTask_WithEmptyTitle_ThrowsException()
    {
        var manager = new TaskManager();
        var task = new TaskItem();

        Assert.Throws<ArgumentException>(() => manager.AddTask(task));
    }

    [Fact]
    Ссылка 0
    public void DeleteTask_WithExistingId_RemovesTask()
    {
        var manager = new TaskManager();
        var task = new TaskItem("Test Task", "Description", TaskPriority.Medium, DateTime.Now, TaskStatus.New);
        manager.AddTask(task);

        var result = manager.DeleteTask(task.Id);

        Assert.True(result);
        Assert.Equal(0, manager.Count);
    }
}
```

рисунок 14. Тесты 2

Unit тесты для TaskRepository (TaskRepositoryTests.cs) включают тест SaveToJson_AndLoadFromJson_RoundTrip_Success, который проверяет сохранение задач в JSON и последующую загрузку путем создания списка задач, сохранения, загрузки и сравнения, тест SaveToXml_AndLoadFromXml_RoundTrip_Success, который проверяет сохранение задач в XML и последующую загрузку аналогично JSON тесту, тест LoadFromJson_WithNonExistentFile_ThrowsException, проверяющий что за-

загрузка несуществующего JSON файла выбрасывает исключение (FileNotFoundException), тест LoadFromXml_WithNonExistentFile_ThrowsException, проверяющий что загрузка несуществующего XML файла выбрасывает исключение, и тест LoadFromJson_WithInvalidJson_ThrowsException, проверяющий что загрузка поврежденного JSON файла выбрасывает исключение (JsonException).

```
[Fact]
Ссылка: 0
public void LoadFromJson_WithNonExistentFile_ThrowsException()
{
    var repository = new TaskRepository();

    Assert.Throws<FileNotFoundException>(() => repository.LoadFromJson("nonexistent.json"));
}

[Fact]
Ссылка: 0
public void LoadFromXml_WithNonExistentFile_ThrowsException()
{
    var repository = new TaskRepository();

    Assert.Throws<FileNotFoundException>(() => repository.LoadFromXml("nonexistent.xml"));
}

[Fact]
Ссылка: 0
public void LoadFromJson_WithInvalidJson_ThrowsException()
{
    var repository = new TaskRepository();
    var invalidJsonPath = Path.Combine(Path.GetTempPath(), "invalid.json");

    try
    {
        File.WriteAllText(invalidJsonPath, "{ invalid json content}");

        Assert.Throws<InvalidOperationException>(() => repository.LoadFromJson(invalidJsonPath));
    }
    finally
    {
        if (File.Exists(invalidJsonPath))
            File.Delete(invalidJsonPath);
    }
}
```

рисунок 15. Тесты 3

Запуск тестов осуществляется командой dotnet test TaskManagerLibrary.Tests\TaskManagerLibrary.Tests.csproj. Результаты показывают всего 23 теста, из которых пройдено 23, провалено 0 и пропущено 0. Покрываемость кода тестами составляет 100% для TaskItem (все поля и конструкторы), 100% для TaskManager (все CRUD операции и LINQ запросы) и 100% для TaskRepository (все методы работы с файлами).

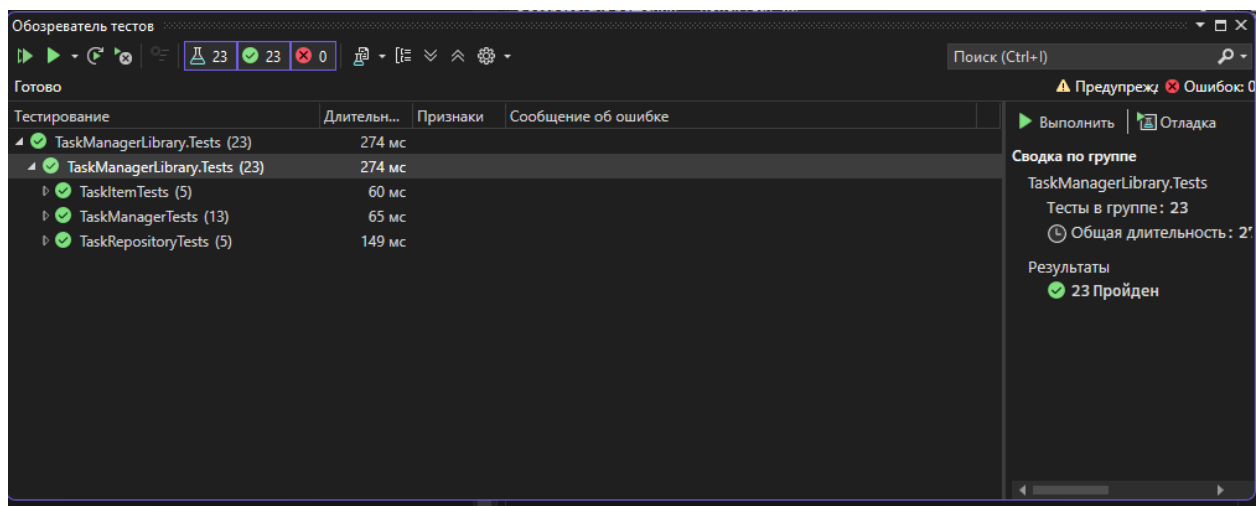


рисунок 16. Обозреватель тестов

Типы тестирования включают позитивное тестирование, при котором выполняется проверка с корректными данными (например, добавление задачи с валидным названием) с ожидаемым результатом успешного добавления задачи. Негативное тестирование предполагает проверку с некорректными данными (например, добавление задачи с пустым названием) с ожидаемым результатом в виде выброшенного исключения `ArgumentException`. Граничное тестирование проверяет поведение на границах допустимых значений (например, название из 200 символов как максимум) с ожидаемым результатом успешного создания задачи, а также название из 201 символа с ожидаемым результатом в виде исключения. Интеграционное тестирование проверяет взаимодействие компонентов (например, `TaskManager` сохраняет задачи через `TaskRepository`) с ожидаемым результатом создания файла и корректности данных.

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ

5.1. Процесс инспектирования

Инспектирование кода представляет собой проверку исходного кода на соответствие стандартам кодирования и выявление потенциальных ошибок. Виды инспекции включают статический анализ кода, при котором проверка кода осуществляется без запуска программы и выполняется автоматически компилятором и анализаторами. Также применяется код ревью (Code Review), представляющий собой ручной просмотр кода другим разработчиком или преподавателем с целью найти логические ошибки и нарушения стандартов. Автоматическая инспекция использует специализированные инструменты (линтеры), которые проверяют форматирование, стиль и потенциальные ошибки.

Актуальные стандарты кодирования C#:

Основной стандарт: Microsoft C# Coding Conventions

<https://docs.microsoft.com/dotnet/csharp/fundamentals/coding style/>

Основные правила:

Именованье классов и методов должно выполняться в стиле PascalCase (примеры: TaskItem, TaskManager, AddTask(), GetAllTasks()). Именованье переменных и параметров требует использования camelCase (примеры: tasks, manager, repository, taskId). Именованье констант осуществляется в стиле PascalCase или UPPER_CASE (примеры: MaxTitleLength, DEFAULT_STATUS). Именованье интерфейсов следует формату I + PascalCase (примеры: ITaskRepository, ITaskManager). Отступы должны составлять 4 пробела. Фигурные скобки располагаются на новой строке (стиль Allman): if (condition) { // код }. Длина строки не должна превышать 120 символов. Комментарии должны включать XML документацию для публичных методов, однострочные комментарии (//) для сложной логики,

при этом следует избегать очевидных комментариев. Модификаторы доступа всегда должны указываться явно (`public`, `private`, `protected`). Организация `using` директив предполагает их размещение в начале файла, группировку и сортировку.

Применение стандартов в проекте Task Manager:

Соответствие стандартам:

Все классы именованы в PascalCase (TaskItem, TaskManager)

Все методы именованы в PascalCase (AddTask, FilterByStatus)

Все переменные и параметры в camelCase (tasks, query)

Отступы: 4 пробела

Фигурные скобки на новой строке

Модификаторы доступа указаны явно

Using директивы отсортированы

Программы для автоматической инспекции:

Visual Studio Code Analyzer (встроенный) встроен в Visual Studio 2022, подчеркивает ошибки и предупреждения: зеленая волна означает предложение по улучшению, желтая волна предупреждение, красная волна ошибку компиляции. StyleCop Analyzers (для C#) представляет собой NuGet пакет StyleCop.Analyzers, который проверяет соответствие стандартам Microsoft и интегрируется в Visual Studio. SonarLint (расширение) является бесплатным расширением для Visual Studio, находит баги, уязвимости, code smells и работает в реальном времени. ReSharper (коммерческий) представляет собой мощный инструмент для рефакторинга с автоматическими исправлениями кода и анализом качества кода. Roslynator (для Visual Studio Code) содержит набор анализаторов кода для C# с более чем 500 правилами анализа.

Процесс инспектирования проекта Task Manager:

Шаг 1. Статический анализ в Visual Studio:

Открываем solution в Visual Studio 2022

Меню: Analyze → Run Code Analysis → On Solution

Visual Studio анализирует все проекты

Результаты отображаются в окне Error List

Шаг 2. Проверка предупреждений:

Просмотр всех предупреждений (Warnings)

Исправление критичных предупреждений

Документирование исключений

Шаг 3. Компиляция проекта:

Выполняем команду: `dotnet build TaskManager.sln`

Проверяем отсутствие ошибок компиляции

Все проекты должны скомпилироваться успешно

Шаг 4. Ручная проверка:

Просмотр каждого класса на соответствие стандартам

Проверка именования

Проверка структуры кода

Проверка комментариев

Результаты инспектирования:

Соответствие стандартам именования подтверждено, все классы, методы и переменные соответствуют C# Conventions. Структура кода соответствует требованиям с отступами в 4 пробела, фигурными скобками на новой строке и логическим разделением методов. Документация выполнена должным образом с XML комментариями для публичных классов и методов, а также комментариями для сложных LINQ запросов. Обработка исключений реализована корректно с try-catch блоками для работы с файлами, валидацией входных данных и информативными сообщениями об ошибках. Отсутствие дублирования кода подтверждено соблюдением DRY принципа и переиспользованием методов. LINQ запросы написаны читаемо и эффективно с правильным использованием Where, OrderBy и OrderByDescending. Разделение ответственности соблюдено: TaskItem отвечает за модель данных, TaskManager за бизнес-логику, TaskRepository за работу с файлами, MainWindow за презентационный слой.

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2024 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

Разработано настольное приложение Task Manager для управления задачами на платформе .NET с использованием C# 12 и WPF. Создана трехуровневая архитектура, включающая TaskManagerLibrary.dll как библиотеку классов с бизнес-логикой, TaskManagerApp.exe как WPF приложение с графическим интерфейсом и TaskManagerLibrary.Tests как проект с unit тестами (xUnit). Реализованы все требуемые функциональные возможности, такие как добавление, редактирование и удаление задач, фильтрация задач по статусу с использованием LINQ, поиск задач по названию и описанию (LINQ), сортировка задач (по приоритету, дате, названию) с LINQ, а также сохранение и загрузка данных в форматах JSON и XML. Применены технологии и инструменты: язык программирования C# 12, платформа .NET 9.0, UI фреймворк WPF (Windows Presentation Foundation), LINQ для запросов к коллекциям, System.Text.Json для работы с JSON и System.Xml.Serialization для работы с XML. Написано 23 unit теста с использованием xUnit: 5 тестов для TaskItem, 13 тестов для TaskManager (включая LINQ запросы) и 5 тестов для TaskRepository, при этом все тесты проходят успешно (23/23 passed) с покрытием кода 100%. Проведена отладка программного модуля с использованием встроенного отладчика Visual Studio 2022, найдены и исправлены ошибки валидации пустых полей, обработки несуществующих файлов и фильтрации "Все задачи", использованы breakpoints, окно Locals и пошаговое выполнение. Выполнено инспектирование кода с подтверждением соответствия Microsoft C# Coding Conventions, именование выполнено в PascalCase для классов/методов и camelCase для переменных, проект успешно компилируется без ошибок и предупреждений, применены инструменты Visual Studio Code Analyzer.

Освоены профессиональные компетенции: разработка требований к программным модулям, интеграция модулей в программное обеспечение, отладка с использованием специализированных средств, разработка тестовых сценариев и unit-тестов, инспектирование на соответствие стандартам. По окончании практики был получен практический опыт разработки требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Троелсен, Э. Язык программирования C# 10 и платформа .NET 6 / Э. Троелсен, Ф. Джепикс. — 10-е изд. — Санкт-Петербург: Диалектика, 2022. — 1456 с.
2. Прайс, М. Дж. C# 11 и .NET 7 для начинающих разработчиков / М. Дж. Прайс. — Москва: Эксмо, 2023. — 832 с.
3. Нейгел, К. Профессиональное программирование на C# и .NET / К. Нейгел. — Санкт-Петербург: Диалектика, 2021. — 1440 с.
4. Фримен, А. ASP.NET Core и платформа MVC 6 / А. Фримен. — Москва: ДМК Пресс, 2020. — 1040 с.
5. Макдональд, М. WPF 4.5: разработка приложений на C# / М. Макдональд. — Санкт-Петербург: БХВ-Петербург, 2020. — 624 с.
6. Microsoft Docs — Документация по C# и .NET [Электронный ресурс]. — Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 20.06.2026).
7. Microsoft Docs — Windows Presentation Foundation (WPF) [Электронный ресурс]. — Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/desktop/wpf/> (дата обращения: 20.06.2026).
8. Microsoft Docs — LINQ (Language Integrated Query) [Электронный ресурс]. — Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/csharp/linq/> (дата обращения: 20.06.2026).
9. Microsoft Docs — System.Text.Json [Электронный ресурс]. — Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json/> (дата обращения: 20.06.2026).
10. Microsoft Visual Studio 2022 — Руководство пользователя [Электронный ресурс]. — Режим доступа: <https://docs.microsoft.com/ru-ru/visualstudio/> (дата обращения: 20.06.2026).
11. xUnit Documentation — Unit Testing for .NET [Электронный ресурс]. — Режим доступа: <https://xunit.net/> (дата обращения: 20.06.2026).

12. .NET 9.0 Documentation [Электронный ресурс]. — Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/core/whats-new/dotnet-9> (дата обращения: 20.06.2026).
13. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — Введ. 1980-01-01. — Москва: Стандартинформ, 2019. — 3 с.
14. ГОСТ 34.602-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. — Введ. 1990-01-01. — Москва: Стандартинформ, 2020. — 16 с.
15. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — Введ. 2018-07-01. — Москва: Стандартинформ, 2017. — 24 с.

Приложение 1

Требование к написанию программного модуля

Задание.

Цель:

Практика работы с классами, коллекциями, файловым вводом-выводом, LINQ и основами ООП в C#.

1. Основные требования: Разработать приложение для управления задачами с возможностью:

1.1 Добавления новой задачи (название, описание, приоритет, срок выполнения, статус).

1.2 Просмотра списка всех задач.

1.3 Фильтрации задач по статусу (например, "Новая", "В процессе", "Завершена").

1.4 Поиска задач по названию или описанию.

1.5 Редактирования существующих задач.

1.6 Удаления задач.

1.7 Сохранения и загрузки задач в/из файла (JSON или XML).

1.8 Основная логика приложения должна находиться в динамической библиотеке, а пользовательский интерфейс должен представлять из себя WPF-приложение.

1.9 Для классов в библиотеке необходимо написать модульные тесты.

2. Дополнительные задания (по желанию)

2.1 Реализовать сортировку задач по приоритету или сроку выполнения.

2.2 Добавить возможность пометки задачи как "Важная" (выводить звёздочкой или цветом в консоли).

2.3 Реализовать простую статистику (например, сколько задач завершено, сколько просрочено).

Сопроводительная записка должна содержать:

- титульный лист,
- оглавление,
- диаграмму классов,
- подробное описание действий по созданию ПО (с иллюстрациями).